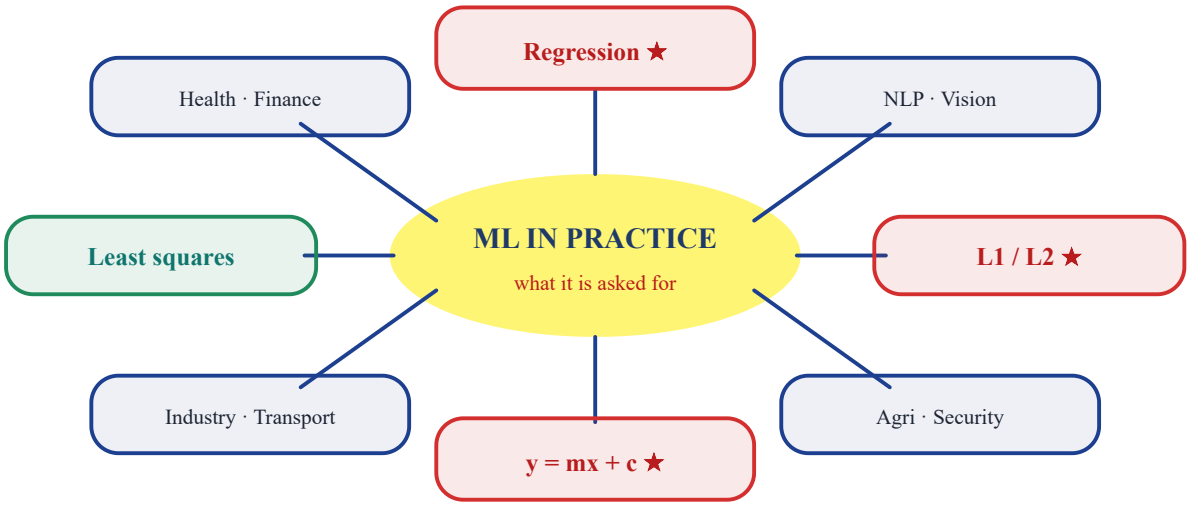
Practical ML Applications & Regression

y = mx + c · least squares · MAE / MSE / RMSE / R² · L1 & L2: 11 sheets



↑ the whole topic on one page: red = high-yield, green = the rule everything is measured by

Syllabus coverage

The AI syllabus lists: old and new AI techniques; knowledge representation; propositional and first-order logic; search; probabilistic reasoning; NLP; expert systems; machine learning; supervised and unsupervised learning; **practical application of machine learning**; **regression**. This notebook takes exactly those last two lines — the three paradigms belong to the ML-fundamentals lesson and Bayes/NLP theory to the probabilistic-reasoning lesson.

High-yield list

① Regression = a number, classification = a label
② $\hat{y} = \beta_0 + \beta_1 x$, that is $y = mx + c$
③ OLS minimises $\Sigma(y_i - \hat{y}_i)^2$
④ MAE = mean$|y - \hat{y}|$ · MSE = mean$(y - \hat{y})^2$
⑤ RMSE = √MSE, in the units of y
⑥ $R^2 = 1 - SSres / SStot$; $R^2 = r^2$
⑦ Overfitting = train ≫ test; fix L1/L2
⑧ $r \in [-1, 1]$ is not causation

Legend: **yellow** = key term · **red circle** = must-memorise · **purple** = memory trick · **red** = exam trap · **green** = worked example

Contents

✓ ML pipeline & task … p. 2
✓ Applications I: health to trade … p. 3
✓ Applications II: text to security … p. 4
✓ Regression & y = mx + c … p. 5
✓ Least squares · metrics … p. 6, 7
✓ Night-before 10 · Practice 11

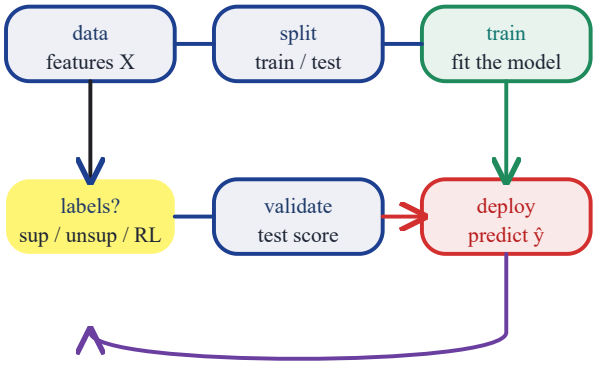
1 ML in Practice: Pipeline & Task

Definition: A machine-learning system is a **function fitted to data**, not a bundle of hand-written rules. Given input features **X**, produce an output **y** — and the kind of output decides the method: **a number is regression**, **a label is classification**, **a group is clustering**, **rewards after actions is reinforcement learning**.

Task	Output	Typical method	One-line example
Regression	real number ŷ	linear / polynomial regression, SVR, GBM	tomorrow's sales
Classification	label + probability	logistic regression, decision tree, SVM, CNN	spam or not spam
Clustering	a group id	k-means, hierarchical, DBSCAN	customer segments
Reinforcement	a policy	Q-learning, SARSA, DQN	a robot learning to walk

Memory trick: One question — **"is the answer a ruler or a yes/no?"** Ruler → regression. Yes/no → classification. Neither → grouping or rewards after acting.

Exam trap: "ML predicts the future" is **wrong**: ML fits a function to **past data** and extrapolates. The test set must stay **unseen**.



drift → retrain · never score on the training data

one pipeline serves every application in the next two sheets

Worked example 1 ✍ label the task: (i) predict the sale price of a flat → **regression**; (ii) decide whether a transaction is fraudulent → **classification**; (iii) group ATMs by usage pattern, no labels → **clustering**; (iv) an agent trades and is paid or fined → **reinforcement learning**.

Worked example 2 ✍ same data, two models: A bank wants the **probability of default** (logistic regression → a number in 0…1) and, once the loan is approved, the **expected loss in taka** (linear regression). One table, two targets.

Why deployment is hard: Offline 0.92 can still fail in production once the input distribution moves — that is **model drift**.

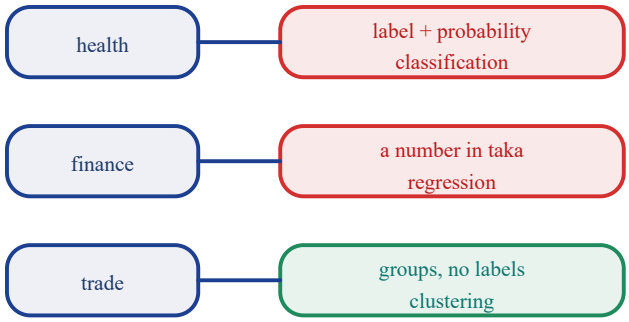
What a feature is: One input column — floor area, age, RMS, a word of the text. A numeric input is what a regressor wants.

2 Applications I: Health, Finance, Trade

Definition: **Practical application of machine learning** means putting a fitted model behind a real decision: a diagnosis, a loan, a shelf, a machine part. Every row below is a **prediction**, and the exam wants the **task** and the **model family**.

Application	What the model does	Task
Health · diagnosis	X-ray or MRI → tumour marked (CNN)	classification
Health · drug discovery	a compound's activity, its toxicity	regression
Health · monitoring	ICU sensors → deterioration	regression
Health · readmission	back again within 30 days?	classification
Finance · credit score	probability of default	classification
Finance · fraud	the fake 0.01 % of payments	unsupervised
Finance · trading	the next price move	regression
Finance · cash demand	what an ATM needs next week	regression
Trade · recommend	"bought X also bought Y"	ranking
Trade · segments	group buyers with k-means	unsupervised
Trade · demand	units to stock next month	regression

Memory trick: Read the **output word** in the stem: price, demand, temperature, amount, risk score → **regression**; spam, fraud, disease, default, defect → **classification**; segment, group, cluster → **unsupervised**. The words **rank** or **recommend** mean the model sorts rather than predicts.



same data, three targets — the output picks the method

Exam trap: Credit scoring is **classification** (will this borrower default?), not regression, even though banks quote a "score". A recommender outputs a **ranking**, not a price. And **fraud data are tiny-class**: anomaly detection trained mostly on normal traffic is the useful model, and accuracy is a useless score.

Worked example ✍ choosing the metric: Fraud data: 99 900 normal, 100 fraudulent. A model answering "normal" for everything scores **99.9 % accuracy and finds zero fraud**. The honest scores are **recall on the fraud class** and the **false positives** an analyst must clear per day.

Where this touches other lessons: The **expert systems** lesson covers the older rule-based alternative to every row above; the **ML fundamentals** lesson defines the three paradigms properly. Here we need only the mapping *sector → task → model family*.

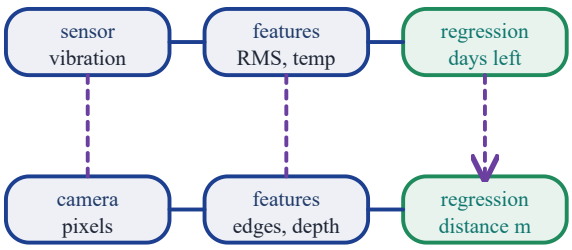
3 Applications II: Text to Security

Definition: The same pipeline with non-tabular data. Text becomes tokens and embeddings, an image becomes pixels, a machine becomes a **sensor time series**, a road becomes a stream of detector counts. The model family changes; the task grammar does not.

Application	Input → output	Task
NLP · sentiment	review → positive / negative	classification
NLP · translation	sentence A → sentence B	mapping
NLP · spam filter	email → keep or quarantine	classification
Vision · self-driving	frames → objects, lanes, distance	detection
Vision · quality control	line-scan image → pass / reject	classification
Industry · maintenance	vibration → days to failure	**regression**
Industry · process	sensors → best set-point	regression
Transport · traffic	loop detectors → speed, flow	regression
Transport · ride-hailing	hour + weather → trips	regression
Agriculture · yield	soil + NDVI → tonnes/ha	regression
Security · intrusion	network flows → normal / attack	unsupervised
Security · malware	binary → malware family	classification

Memory trick: "Image or sensor in, **a number out** for predictive maintenance and traffic." Whenever the stem says *how much / how many / how long / when will*, you are in regression — even if the input was a photograph.

Exam trap: Predictive maintenance is **regression** (how many days of life are left) while **fault detection** is classification (is it faulty now?) — two models, and examiners do swap them. In "intrusion detection", **unsupervised** usually means trained on normal traffic only, so a *novel* attack is still caught.



the middle box is the same for every sector
the third box is the exam answer

unstructured input, engineered features, numeric output

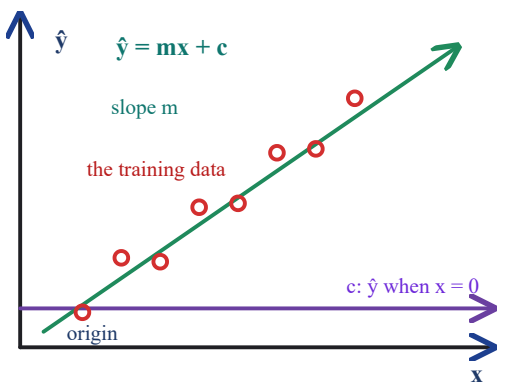
Worked example ✍️ maintenance, end to end: A bearing's vibration RMS grows almost linearly from 0.4 to 1.8 mm/s over the last 400 hours before failure. Fit **remaining life = 400 − 220 × RMS**. At RMS = 1.2 mm/s the model warns **136 hours** ahead; at 1.6 mm/s, **48 hours**. The failure is called *before* it happens.

4 Regression & the Model y = mx + c

Definition: **Regression** is **supervised learning for a continuous numerical output**. With one feature it is the **simple linear regression** line **$\hat{y} = m\,x + c$**: m is the **slope** (change in ŷ per unit x), c the **intercept** (ŷ when x = 0). With p features it becomes **multiple linear regression**: $\hat{y} = \beta_0 + \beta_1 x_1 + \ldots + \beta_p x_p + \varepsilon$.

c is ŷ at x = 0, nothing more

Point of comparison	Regression	Classification
Output	any real number, unbounded	one of a fixed label set
Typical loss	squared error (MSE), MAE	cross-entropy, 0–1 loss
Typical scores	**MAE, MSE, RMSE, R^2**	accuracy, precision, recall, F1
Baseline to beat	predict the mean of y	predict the majority class
Symbolic form	$\hat{y} = mx + c$	$P(Y=1\|X) = 1/(1 + e^{-z})$
Examples	house price, temperature	spam or not spam



the model is the green line; the red dots are what it must fit

Derivation sketch ✍️ why the line passes through ($\bar{x}$, $\bar{y}$):

1. Least squares picks m, c so the vertical errors are smallest
2. Both derivatives zero ⟹ $\Sigma\hat{y} = \Sigma y$ ⟹ $c = \bar{y} - m\,\bar{x}$, so the line **always** threads **($\bar{x}$, $\bar{y}$)**

The two numbers you need: $m = \Sigma(x_i - \bar{x})(y_i - \bar{y}) / \Sigma(x_i - \bar{x})^2$ · $c = \bar{y} - m\,\bar{x}$, and $m = r \cdot \sigma_y/\sigma_x$ takes the sign of r.

Memory trick: **"m moves, c anchors."** m answers *how much does ŷ change when x rises by 1?*; c answers *what is ŷ at x = 0?* The fitted line always threads the mean point ($\bar{x}$, $\bar{y}$) — the fastest way to see whether a candidate answer is plausible.

Exam trap: (i) The intercept is **ŷ at x = 0**, not the mean of y. (ii) **Logistic regression is classification** despite the name. (iii) A best-fit line is **not** required to pass through the origin. (iv) Regression needs no **balanced** class structure.

Worked example ✍️ read the model, then predict: A flat-price model in lakh BDT is **$\hat{y}$ = 18.5 + 3.2 × (area in m²)**. For 90 m², $\hat{y}$ = 18.5 + 288 = **306.5** lakh. Each extra m² adds **3.2 lakh** (m = 3.2), while a zero-area plot would be valued at **18.5 lakh** (c = 18.5) — really the price of the land and registration.

5 Least Squares (OLS)

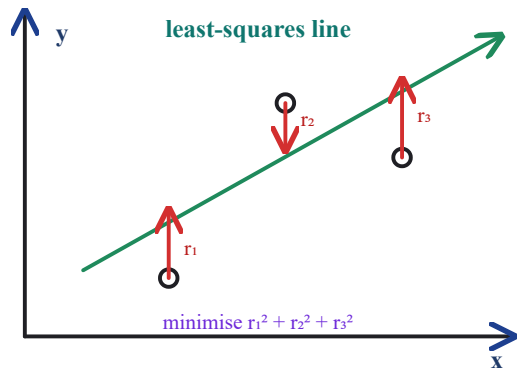
Definition: **Ordinary least squares (OLS)** chooses the coefficients that minimise the **sum of squared residuals**: minimise SSE = $\Sigma(y_i - \hat{y}_i)^2$ over every training point. Counting wrong answers belongs to classification; OLS never counts — it **squares and adds**.

Derivation ✍️ the normal equations:

1. Objective: $SSE(a, b) = \Sigma(y_i - a - b\,x_i)^2$
2. Partials zero: $\partial SSE/\partial a = -2\Sigma(y_i - a - b\,x_i) = 0$ and $\partial SSE/\partial b = -2\Sigma x_i(y_i - a - b\,x_i) = 0$
3. Two **normal equations**: $na + b\,\Sigma x = \Sigma y$ and $a\,\Sigma x + b\,\Sigma x^2 = \Sigma xy$
4. Solving: $\mathbf{b = [n\Sigma xy - (\Sigma x)(\Sigma y)] / [n\Sigma x^2 - (\Sigma x)^2]}$, $\mathbf{a = \bar{y} - b\,\bar{x}}$

x	1	2	3	Σ
y	2	4	5	11
xy	2	8	15	25
x^2	1	4	9	14

x	1	2	3	Totals
actual	2.000	4.000	5.000	$\Sigma y = 11$
fitted $\hat{y}$	2.167	3.667	5.167	$\Sigma\hat{y} = 11$
residual	−0.167	+0.333	−0.167	$\Sigma e = 0$
e^2	0.028	0.111	0.028	SSE = **0.167**



the residuals $r_i = y_i - \hat{y}_i$ are squared and added — never counted

Answer for (1, 2), (2, 4), (3, 5): n = 3, Σx = 6, Σy = 11, Σxy = 25, Σx^2 = 14 ⟹ **b = [3(25) − 6(11)] / [3(14) − 6²] = (75 − 66) / (42 − 36) = 9/6 = 1.5** and **a = 11/3 − 1.5(2) = 2/3 ≈ 0.67**, so **$\hat{y}$ = 0.67 + 1.5x**.

Memory trick: "**O**ne line, **L**east **S**quares, so the **S**um is what shrinks." Check the giveaway: on a correct OLS fit the residuals **sum to zero** and the fitted values **sum to the actuals**, because the line passes through ($\bar{x}$, $\bar{y}$).

Exam trap: OLS is called **ordinary** because it weights every point equally — **weighted least squares** is the variant that does not. Squaring means one huge residual **dominates**, so with outliers use MAE-fitting, Huber loss or quantile regression.

IT link ✍ the same maths in code:

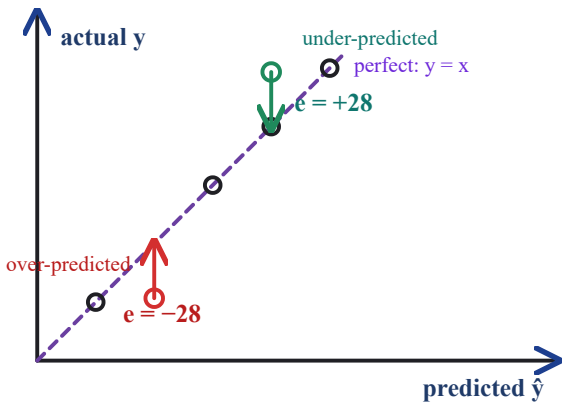
```
b = (n*Sxy - Sx*Sy) / (n*Sxx - Sx^2);  a = ybar - b*xbar
```

6 MAE, MSE, RMSE and R^2

Definition: Every metric is built from the residual $\mathbf{e_i = y_i - \hat{y}_i}$. MAE and MSE average the error in two different ways, RMSE is simply MSE brought back into the units of y, and **R^2 compares the model against the "always predict the mean" baseline**.

Metric	Formula	Property and when to use it
MAE	$(1/n)\,\Sigma\lvert y_i - \hat{y}_i\rvert$	mean absolute error: in the units of y; **robust** — outliers barely matter
MSE	$(1/n)\,\Sigma(y_i - \hat{y}_i)^2$	mean squared error: units are y^2; **punishes large errors much harder**; trains the model
RMSE	$\sqrt{\text{MSE}}$	root mean squared error: back in the units of y; "off by X units on average"
R^2	$1 - SS_{res}/SS_{tot}$	coefficient of determination: fraction of variance explained; **can be negative**; $R^2 = r^2$ for a simple line

Worked example 1 ✍ the OLS fit of sheet 6: residuals −0.167, +0.333, −0.167: **MAE** = 0.667/3 = **0.222** · **MSE** = 0.167/3 = **0.0556** · **RMSE** = √0.0556 = **0.236** in the units of y. SS_{res} = 0.167, SS_{tot} = 2.778 + 0.111 + 1.778 = 4.667, so **$R^2 = 1 - 0.167/4.667 =$ 0.964**: the line explains 96.4 % of the variation in y.



every dot is a prediction; the gap is the residual, and SSE sums its square

Relations worth knowing: **MAE ≤ RMSE** always · **MSE ≥ MAE²**, with equality only when every residual has the same size · SStot = SSreg + SSres · R^2 = SSreg/SStot · adjusted $R^2 = 1 - (1 - R^2)(n - 1)/(n - p - 1)$.

Worked example 2 ✍ one outlier flips MSE: Four residuals 10, 10, 10, 10 give MAE = 10, MSE = 100. Change the last to 40: MAE = 17.5, MSE = 475, RMSE = 21.8 — **MAE rose 75 %, MSE rose 375 %.**

Memory trick: "**Square, average, root.**" MSE = average of squares, RMSE = root of that average, so RMSE = √MSE and it shares the units of y. For R^2: **1** perfect, **0** = no better than the mean, **negative** = worse.

Exam trap: R^2 is not a percentage and can be **negative**. MSE is in **y^2**, so only RMSE is readable. MAE is not "more accurate", only differently behaved.

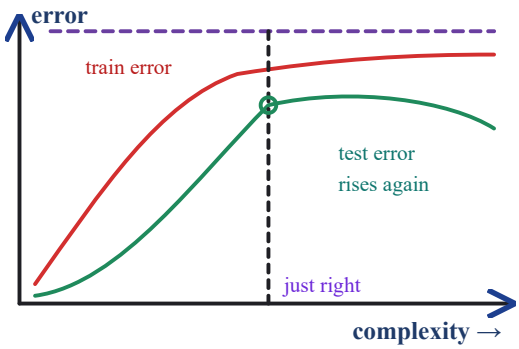
7 Overfitting, Bias–Variance, L1/L2

Definition: **Overfitting** is fitting the training data so closely — noise included — that the model fails on unseen data. The symptom is the **train–test gap**: 99 % on training, 60 % on test, formally **high variance**. The opposite failure, poor scores on *both*, is **underfitting = high bias**.

Symptom	Train	Test	Diagnosis
memorising noise	99%	60%	**overfitting**, high variance
model too simple	58%	55%	underfitting, high bias
healthy fit	88%	86%	generalising ✓

Method	Penalty added to the loss	Effect on the coefficients
L2 ridge	$\lambda \Sigma\beta^2$	shrinks every coefficient, **none exactly 0**
L1 lasso	$\lambda \Sigma\lvert\beta\rvert$	sets some coefficients **exactly 0 → feature selection**
Elastic Net	$\lambda_1\Sigma\lvert\beta\rvert + \lambda_2\Sigma\beta^2$	L1's sparsity with L2's stability

Ridge and lasso objective: minimise $\Sigma(y_i - \beta_0 - \Sigma\beta_j x_j)^2 + \lambda\,\Sigma\beta_j^2$ (L2) or $+\,\lambda\,\Sigma\lvert\beta_j\rvert$ (L1). A bigger λ means heavier shrinkage, more bias and less variance; the intercept β_0 is not penalised.



train error falls for ever; test error turns back up

Memory trick: **"Lasso zeroes, ridge shrinks."** L1's absolute-value penalty is kinked at zero, so a coefficient that is not earning its place is pushed all the way to 0 and its feature is dropped; L2 only ever pulls towards 0.

Exam trap: (i) Overfitting is **high variance**, underfitting is **high bias**. (ii) A larger λ = more regularisation, more bias, less variance. (iii) Regularisation adds no information. (iv) Only **L1** selects features.

Worked example ✍ reading the gap:
Train 99 %, test 60 % ⟹ a 39-point gap ⟹ overfitting. Fix in order of cost: **(1)** more data, **(2)** L2 / L1, **(3)** cross-validation, **(4)** a simpler model.

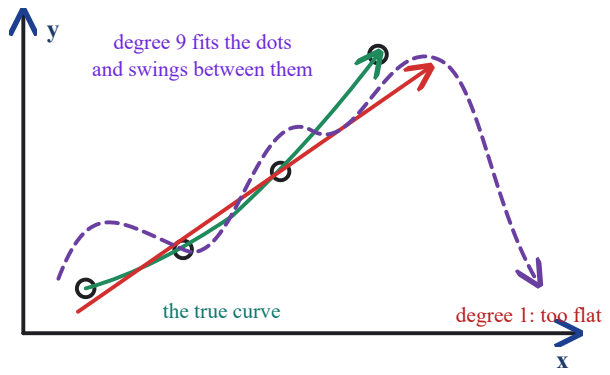
Only a held-out set tells you:
A test set — or k-fold cross-validation — exposes the gap; a common split is 60 / 20 / 20.

8 Polynomial & Correlation ≠ Causation

Definition: **Polynomial regression** keeps the linear least-squares machinery but adds powers of the feature: $\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2 + \ldots + \beta_n x^n$. **Correlation** is the standardised strength of a *linear* association, Pearson's $r = \text{cov}(x, y)/(\sigma_x \sigma_y)$, and it always lies in **[−1, +1]**.

Model	Form	Shape and main risk
Linear	$\hat{y} = mx + c$	one straight line; cannot bend, so it underfits curved data
Polynomial, degree n	$\hat{y} = \beta_0 + \beta_1 x + \ldots + \beta_n x^n$	at most n − 1 turning points; **overfits** and oscillates between points
Logistic	$1/(1 + e^{-z})$	S-shaped probability in 0…1; classification only, boundary linear in x

Worked example ✍️ a line under-fits, a curve nails it:
Data (0, 1), (1, 3), (2, 7), (3, 13). Second differences are 2, 2 — constant, so the data are exactly quadratic: $\hat{y} = 1 + x + x^2$, SSE = **0**. The **linear** fit gives b = 20/5 = 4, a = 0, so $\hat{y} = 4x$ with residuals **+1, −1, −1, +1**: MAE = 1, MSE = 1, $R^2 = 1 - 4/84 = 0.952$.



more power always lowers training error — and can wreck the model

Memory trick: A linear fit that is **wrong but not random** leaves a residual pattern: + − − + is a curve, **random signs** is noise.

Correlation is not causation: r = 0.98 between ice-cream sales and drownings, and neither causes the other: both are driven by a **confounding variable** — hot weather. r says nothing about **direction** or non-linearity, and can vanish per individual (**Simpson's paradox**).

Exam trap: (i) r = 0.9 does **not** prove causation. (ii) r = 0 does **not** prove independence — it rules out only a **linear** relation. (iii) For a **simple** line $R^2 = r^2$ exactly. (iv) A degree-n polynomial bends at most **n − 1** times.

Worked example ✍️ two coefficients, one data set:
For the same four points: $r = 20/\sqrt{(5 \times 84)} = \mathbf{0.976}$ and the linear $R^2 = r^2 = 0.952$. Add x^2 and R^2 becomes 1.000 while r is unchanged.

Night-Before-the-Exam Sheet

Formula sheet:

Model $\hat{y} = mx + c$ (m slope, c intercept)

General $\hat{y} = \beta_0 + \Sigma\beta_j x_j + \varepsilon$

Fit passes through $(\bar{x}, \bar{y})$

$\mathbf{m} = \Sigma(x-\bar{x})(y-\bar{y}) / \Sigma(x-\bar{x})^2$

$\mathbf{m} = [n\Sigma xy - \Sigma x\Sigma y] / [n\Sigma x^2 - (\Sigma x)^2]$

$\mathbf{c} = \bar{y} - m\,\bar{x}$

Residual $e = y - \hat{y}$ · **OLS** minimises Σe^2

MAE = mean|e| · **MSE** = mean(e^2)

RMSE = √MSE, units of y

$\mathbf{R^2} = 1 - SSres/SStot$

Adjusted $\mathbf{R^2} = 1 - (1-R^2)(n-1)/(n-p-1)$

$\mathbf{r} = cov/(\sigma x \sigma y) \in [-1, 1]$

Simple line $R^2 = r^2$ · **L2** $+ \lambda\Sigma\beta^2$

L1 lasso $+ \lambda\Sigma|\beta|$

Logistic $1/(1 + e^{-z})$

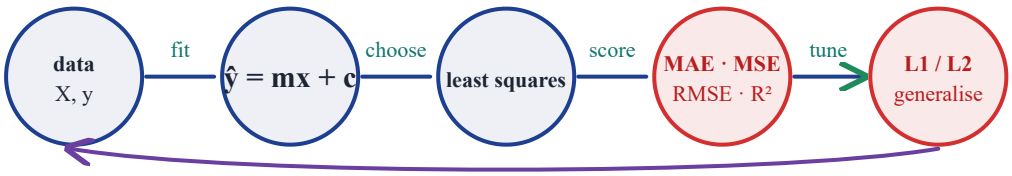
Always MAE ≤ RMSE, MSE ≥ MAE^2

Top 12 one-liners for MCQs

1. Regression predicts a **continuous number**
2. Classification predicts a **label** and a probability
3. k-means with no labels = **unsupervised**
4. Reward after acting = **reinforcement learning**
5. OLS minimises $\mathbf{\Sigma(y - \hat{y})^2}$, never a count
6. c is $\mathbf{\hat{y}}$ **at x = 0**, not the mean of y
7. RMSE = √MSE, in the units of y
8. MAE is robust; MSE punishes big errors
9. R^2 can be **negative**
10. Overfitting = **high variance**, train ≫ test
11. L1 lasso **zeroes** features, L2 ridge shrinks
12. r ∈ [−1, 1]; correlation ≠ causation

Top traps: logistic regression is classification · the intercept is not $\bar{y}$ · MSE is in y^2, only RMSE is readable · R^2 is not a percentage · bigger λ = more shrinkage = more bias · lasso selects features, ridge does not · a degree-n polynomial has n − 1 turning points · fraud and intrusion data are tiny-class, so accuracy means nothing · the test set must be unseen · r = 0 does not mean independent.

All tricks together: "Ruler or yes/no?" picks regression vs classification · "m moves, c anchors" · the fit threads $(\bar{x}, \bar{y})$ and Σresidual = 0 · "square, average, root" for MSE and RMSE · "lasso zeroes, ridge shrinks" · a residual pattern of + − − + means bend the model, not blame it · two things moving together can both be driven by a third.



data in, four numbers out, then the same four choices every time

✓ Practice MCQs (10 marks)

1. Regression predicts:
(a) a cluster id (b) a real number
(c) a yes/no label (d) a best move

2. OLS minimises:
(a) wrong answers (b) $\Sigma(y - \hat{y})^2$
(c) the sum of x (d) correlation r

3. Least upset by one outlier:
(a) MAE (b) MSE
(c) RMSE (d) none of them

4. For a simple line, R^2 is:
(a) the slope (b) SSres/SStot
(c) the mean $\bar{y}$ (d) r squared

5. L1 (lasso) can:
(a) shrink equally (b) ignore labels
(c) zero some β (d) add no penalty

6. Train 99 %, test 60 %:
(a) overfitting (b) underfitting
(c) the optimum (d) rate too small

7. True of correlation:
(a) r proves cause (b) linear only
(c) needs r = 1 (d) p-value proves

8. MAE = 4 $\Longrightarrow$ MSE is:
(a) ≤ 4 (b) = 4
(c) < 16 (d) ≥ 16

9. Logistic regression predicts:
(a) a sigmoid label (b) a real price
(c) clusters (d) fewer features

10. A polynomial adds:
(a) no extra term (b) no overfit risk
(c) x^2, x^3 … terms (d) images only

Answer key → 1-b · 2-b · 3-a · 4-d · 5-c · 6-a · 7-b · 8-d · 9-a · 10-c

Written question: Fit an ordinary least-squares line to (1, 2), (2, 4), (3, 5). Give the slope, the intercept, the fitted values, MAE, MSE, RMSE and R^2, and comment on the fit. 10 marks

1. **Set-up (2)** — n = 3; $\Sigma x = 6$, $\Sigma y = 11$, $\Sigma xy = 25$, $\Sigma x^2 = 14$; minimise SSE = $\Sigma(y_i - a - b\,x_i)^2$. 2
2. **Coefficients (3)** — $b = [n\Sigma xy - (\Sigma x)(\Sigma y)]/[n\Sigma x^2 - (\Sigma x)^2] = (75 - 66)/(42 - 36) = \mathbf{1.5}$; $a = \bar{y} - b\,\bar{x} = 11/3 - 3 = \mathbf{2/3 \approx 0.67}$, so $\hat{\mathbf{y} = 0.67 + 1.5x}$. 3
3. **Fitted values (2)** — $\hat{y} = 2.167, 3.667, 5.167$; residuals e = −0.167, +0.333, −0.167. Check: $\Sigma e = 0$ and $\Sigma\hat{y} = \Sigma y$, since the line passes through $(\bar{x}, \bar{y})$. 2
4. **Metrics (2)** — MAE = 0.667/3 = **0.222**; MSE = 0.167/3 = **0.0556**; RMSE = $\sqrt{0.0556}$ = **0.236** in the units of y. 2
5. **R^2 and comment (1)** — SSres = 0.167, SStot = 4.667, $R^2 = 1 - 0.167/4.667 = \mathbf{0.964}$: 96.4 % of the variance in y is explained and there is no train–test gap at this size. 1

Before the exam ✓

- ✓ Can label any application regression / classification / clustering / RL
- ✓ Can compute m and c by both formula routes
- ✓ Can compute MAE, MSE, RMSE, R^2 from a residual list
- ✓ Knows why R^2 can be negative and why r ≠ causation